

Fig. 4: Raspberry pin configurations

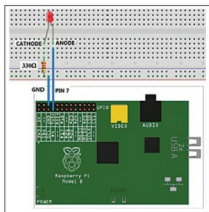


Fig. 5: LED connection to GPIO04/PIN7

```
root@raspberrypi:/sys/class/gpio/gpio4# cat direction
in
root@raspberrypi:/sys/class/gpio/gpio4# echo "out" > direction
root@raspberrypi:/sys/class/gpio/gpio4# cat direction
out
root@raspberrypi:/sys/class/gpio/gpio4# echo 1 > value
root@raspberrypi:/sys/class/gpio/gpio4# cat value
1
root@raspberrypi:/sys/class/gpio/gpio4# echo 0 > value
root@raspberrypi:/sys/class/gpio/gpio4# cat value
0
root@raspberrypi:/sys/class/gpio/gpio4# ls
active_low direction edge power subsystem uevent value
root@raspberrypi:/sys/class/gpio# ls
export gpio4 gpiochip0 unexport
root@raspberrypi:/sys/class/gpio# echo 4 > unexport
root@raspberrypi:/sys/class/gpio# ls
export gpiochip0 unexport
root@raspberrypi:/sys/class/gpio#
```

Fig. 6: Steps for accessing the GPIO and enabling the LED to blink

user space. This is equivalent to enabling the GPIO pin.

Example: `$ echo 19 > export`

This will create a 'gpio19' node (folder) for GPIO #19. This folder contains files which can be read/written to access GPIO 19 pin.

unexport. Reverses the effect of exporting.

Example: `$ echo 19 > unexport`

This will remove a 'gpio19' node (folder).

Now a user can access GPIO using its attributes. To see its attributes, type the below-mentioned commands. (Fig. 3 shows an example of gpio4.)

```
$ cd gpio19
```

```
$ ls
```

After the 'ls' command, you will see below-mentioned attributes:

direction. Reads as either 'in' or 'out' to declare pin as input or output.

value. Reads as either '0' (low) or '1' (high). If the GPIO is configured as an output, this value may be written; any non-zero value is treated as high.

edge. Reads as either 'none', 'rising', 'falling' or 'both.' Write these strings to select the signal edge(s).

active_low. Reads as either '0' (false) or '1' (true). Write any non-zero value to invert the value attribute both for reading and writing.

Now connect the LED as shown in Fig. 5 and follow the steps below to access the GPIO pins and enable the LED to blink (refer Fig. 6 for the steps). We will only use two attributes: direction and value.

1. Start the terminal window on Raspberry Pi and go to `cd /sys/class/gpio/` directory.
2. Login as root (as we are accessing kernel space) by 'su' command and type the root password when prompted.
3. Export GPIO4 using 'echo 4 > export' command. By this statement you reserve GPIO for your application so other modules will not interfere.
4. New directory named 'gpio4' will be created.
5. Enter the directory by typing 'cd gpio4' to see and edit attributes of GPIO4.
6. Change attributes as: `echo "out" > direction`.
7. To switch the LED on use 'echo 1 > value' and to switch LED off use 'echo 0 > value'.
8. To see attribute's value, use cat command: `cat value` or `cat direction`.
9. For disabling the GPIO, first write 'cd .' to come one step back and then use 'echo 4 > unexport' command. The gpio4 folder will be deleted automatically.

By using these steps, you can enable the LED to blink manually. But what if you want it to blink automatically? For that you will use shell scripting.

Shell is a program that runs in background of a terminal. Terminal accepts command from keyboard and sends it to shell, which finds the meaning and executes the command. User enters commands one by one and they are executed in a similar manner. These commands can be stored as a sequence of commands to a file and shell can execute this file instead of entering the commands one by one in the future. This file is known as a shell script. Shell script is a series of commands written in plain text file. Shell script is just like the MS-DOS batch file.

Create a new file by typing 'sudo nano firstscript' in the terminal and enter the code given below in it:

```
#!/bin/bash

cd /sys/class/gpio
echo 4 > export
cd gpio4
echo "out" > direction

for i in {1..10}
do
echo 0 > value
sleep 1
echo 1 > value
sleep 1
done

cd ..
echo 4 > unexport
```

Now save the script by pressing 'ctrl + o' to save and 'ctrl + x' to exit.

To make the file executable, write the below-mentioned command in the terminal window:

```
$ chmod 777 firstscript
Now execute the script as follows
$ sudo ./firstscript
```

Accessing GPIO using SYSfs interface from application

The above steps are for controlling or accessing one GPIO but we can also handle more GPIOs at the same time by using an application. The application ('C' program) that we have discussed here also controls one GPIO, but the